

Where package documentation comes from

Every source, what it really is, and how to get it for a particular version — RubyGems and NuGet

We have 2,814 Ruby gems and 3,352 NuGet packages, and between them 471,930 published versions. For any one of those versions we want its documentation. This explains where it can come from and which sources are worth using.

Ruby: the six places to look

Source	What it actually is	Old versions?	Have it
Doc comments	Comments written above each method, inside the code	Yes	76% of gems
README	A file shipped in the gem archive	Yes	76%
docs/ folder	Guide files shipped in the gem archive	Yes	5%
rubydoc.info	A website that runs YARD over the gem and publishes the result	Yes	93%
GitHub docs/	A folder in the source repository	If tagged	1%
A project website	A site the maintainers wrote themselves	Usually not	20%

What YARD does

YARD reads the comments in the code and turns them into web pages. rubydoc.info runs it over every gem published, automatically. Nobody asks for this and nobody can decline it.

So a rubydoc.info page existing means nothing on its own. If the author wrote no comments, the page still appears — it just lists method names with nothing beside them.

532 gems have a rubydoc.info page and not one documented method. That is why we measure the comments in the code rather than counting whether a documentation link exists.

What “48% of methods documented” means

We look at each method and ask whether a comment sits directly above it:

```
# Parses the config file and returns a Rack app. ← a doc comment
def parse_file(path)
  end
  ← documented

def helper(x)
  end
  ← not documented
```

Across all gems: **453,995 of 938,947 methods carry a comment — 48%**. The median gem manages only 24%.

C#: the four places to look

Source	What it actually is	Old versions?	Have it
XML doc file	The compiler turns /// comments into a file shipped beside the assembly	Yes	65% of packages

Source	What it actually is	Old versions?	Have it
README	A file shipped inside the .nupkg	Yes	35%
GitHub docs/	A folder in the source repository	If tagged	small
A project website	Usually a landing page rather than documentation	Usually not	37%

The C# compiler does the same job YARD does, but its output is a file placed inside the package rather than pages on a website. Each public member gets a structured entry:

```
<member name="M:Serilog.Log.Information(System.String)">
  <summary>Write a log event at Information level.</summary>
  <param name="messageTemplate">The message template.</param>
</member>
```

Two differences from Ruby matter. The file **ships with the package**, so it is versioned automatically and needs no website. But it is **optional** — the author must switch on a build setting, and 35% never do. For those packages nothing is generated at all, and there is no NuGet equivalent of rubydoc.info to fall back on.

How to get documentation for a particular version

Three of the sources come free with the package. Download the version you want and its documentation is already inside.

Ruby — one download per version

```
curl -O https://rubygems.org/downloads/rack-3.2.7.gem
tar -xf rack-3.2.7.gem      # gives data.tar.gz
tar -xzf data.tar.gz        # comments, README and docs/ are all in here
```

C# — read only the part you need

```
curl -O https://api.nuget.org/v3-flatcontainer/serilog/4.4.0/serilog.4.4.0.nupkg
unzip serilog.4.4.0.nupkg  # XML file sits beside each assembly
```

A .nupkg is a zip, and a zip keeps its index at the end. One ranged request reads that index, a second reads just the XML file. That is how 3,352 packages were surveyed while transferring 298 MB instead of 14 GB.

GitHub docs folder — only if the release was tagged

```
https://api.github.com/repos/<owner>/<repo>/contents/docs?ref=v1.2.3
```

This works when the project tagged that release. Gem version 1.2.3 might be tagged v1.2.3, 1.2.3, rel-1.2.3, or not tagged at all — so try each and record which one matched. Monorepos are a further catch: rails/rails tags once for seven gems, so the tag exists but its docs folder is not specific to the gem you asked about.

A project website — usually only one version

bundler.io documents whatever bundler is today. There is normally no way to see what it said three years ago. A few sites keep old versions at addresses like /v2.4/, but they are the exception.

So a website counts for one version only. Every other version of that gem is marked “documented somewhere, version unknown” — not counted as documented, and not counted as missing. Saying today's bundler.io documents bundler 1.16 would claim that version had features it did not have.

The recipe

Step	Do this	Covers
1	Download each version and read what is inside it	96% of gems, 75% of NuGet packages
2	For the rest, check GitHub at the release tag	a small number more
3	For what is still left, record the website against one version	~3%
4	Record the remainder as not found	~1%

Never merge those four into a single percentage. Step 1 is exact evidence; step 3 is one version's worth of evidence stretched over many. Reporting them together would hide which is which.

What it costs to do every version

How many versions	Downloads	Time	Disk
Newest only (what we have now)	6,166	~30 min	10 GB
One per major version	~25,000	~2 h	~40 GB

How many versions	Downloads	Time	Disk
One per minor version ← recommended	~90,000	~7 h	~120 GB
Every version	471,930	~60 h	up to 1.3 TB

Patch releases are bug fixes and their documentation is nearly always identical, so a full run mostly confirms that 1.2.3 and 1.2.4 match. Documentation changes when the API changes, and that happens at minor versions.

Things you cannot get

- **A website's older versions.** Once a site is updated the previous text is gone, unless someone archived it.
- **Documentation for a version that was never published.** Yanked releases disappear from the registry.
- **C# coverage as a percentage.** The compiler records only what was written, so there is no count of undocumented members to divide by. Ruby's 48% has no C# counterpart.

All figures are measured from the 2,814 gems and 3,352 NuGet packages selected in this repository.